Elise C.

3/3/2026

Foundations of Python Programming

Assignment 06

GitHubURL

Functions

Introduction

This week I learned to restructure my code to include classes and functions. This document will detail my process in creating a Python program that went beyond the while loop, programming menus, and conditional logic included in prior assignments. In Assignment 05 we utilized data processing using dictionaries, json files and exception handling. Assignment 06 adds the use of functions, classes, and the separation of concerns pattern.

Preparing to Write

First, I created a new project called “Module 06” in PyCharm IDE. Then I opened the starter file with the instructor’s code from Assignment 05. I saved the file as instructed by the homework document and revised the script header. I referred back to demos and labs to help me understand how the structure of the code must change; this past week was the first time I encountered classes and functions.

Starting the Writing of the Program

This week’s assignment only had two constants and the number of variables were reduced down to two. I created two new classes and seven new functions. At first it took some time to understand how the script needed to be reorganized. It was important to learn how the code should be modified to successfully create classes and functions. I needed to drastically change the code to use the separation of concerns pattern within this new structure of classes and functions. I created descriptive document strings for each class and function.

Testing and Modifying the Code

Once I reviewed the code a few times, I had corrected each potential issue except one warning. For this warning PyCharm displayed a yellow squiggly line beneath “menu_choice” in the last

block of code (see Figure 1). I compared my code against the original lab 3 script file. I could not find any differences that would explain why I had this warning: “Redeclared ‘menu_choice’ defined above without usage.” After trying small changes to match the lab code such as removing the “str” in the variable list and changing the order of two functions, I was not able to change the warning. Reviewing the code numerous times helped me understand the logic and how the structure mapped to different components to make the program work (see Figure 1).

Figure 1. Final Section of Code in PyCharm

```
    # Repeat the following tasks
while True:
    IO.output_menu(menu=MENU)

    menu_choice = IO.input_menu_choice()

    if menu_choice == "1":  # Get new data (and display the change)
        students = IO.input_student_data(student_data=students)
        continue

    elif menu_choice == "2":  # Display current data
        IO.output_student_courses(student_data=students)
        continue

    elif menu_choice == "3":  # Save data in a file and Display data
        FileProcessor.write_data_to_file(file_name=FILE_NAME, student_data=students)
        IO.output_student_courses(student_data=students)
        continue

    elif menu_choice == "4":  # End the program
        break  # out of the while loop
```

Since I had tried everything I could think of to fix the warning, I ran the program (see Appendix for full final code). All of the menu options ran smoothly. I checked the json file and one new student had been added to the file. I had added “indent=2” within the write function to make the json file format more readable.

I ran the program again in PyCharm and I entered one more student (see Figures 2 and 4). I checked the json file and all the data was reformatted. When I ran it in the terminal, it also worked as expected. In the terminal I entered two students to confirm that my program worked for multiple entries (see Figure 4). I also tested the error handling using PyCharm. I moved the Enrollments.json file into another folder and got the error “No such file or directory: 'Enrollments.json'”. Then I tested entering numbers in the first name and the last name prompt. I received the “That value is not the correct type of data!” error. The error testing went smoothly.

Final Program

I created a program that ran well in PyCharm (Figure 2) and the terminal (Figure 3). I was also able to add two new students to my json file (see Figure 4).

Figure 2: PyCharm Output Entering the Second New Student

```
---- Course Registration Program ----

  Select from the following menu:

    1. Register a Student for a Course.

    2. Show current data.

    3. Save data to a file.

    4. Exit the program.

----------------------------------------


Enter your menu choice number: 1

What is the student's first name? Audrey

What is the student's last name? Clark

Please enter the name of the course: Python 100
```

```
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------

Enter your menu choice number: 2

--------------------------------------------------
Bob, Smith, Python 100
Sue, Jones, Python 100
Larry, Larson, Python 100
Audrey, Clark, Python 100
--------------------------------------------------

---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------

Enter your menu choice number: 3
```

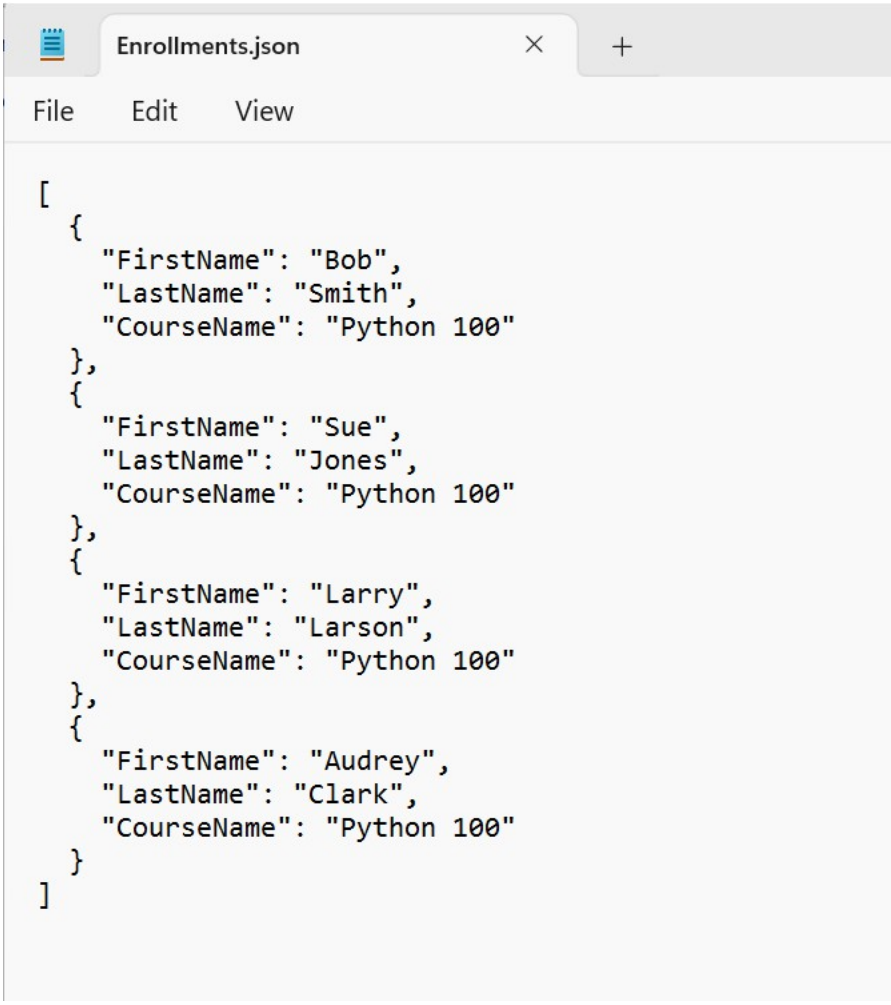
```
--------------------------------------------------
Bob, Smith, Python 100
Sue, Jones, Python 100
Larry, Larson, Python 100
Audrey, Clark, Python 100
--------------------------------------------------

---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------

Enter your menu choice number: 4
Process finished with exit code 0
```

Figure 3. Command Prompt Output Entering the First Student's Data

```
Command Prompt

Microsoft Windows [Version 10.0.26200.7840]
(c) Microsoft Corporation. All rights reserved.

C:\Users\elise>CD Documents\Python\PythonCourse

C:\Users\elise\Documents\Python\PythonCourse>Python Assignment06.py


---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------


Enter your menu choice number: 1
What is the student's first name? Larry
What is the student's last name? Larson
Please enter the name of the course: Python 100
```

```
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------


Enter your menu choice number: 2

--------------------------------------------------
Bob, Smith, Python 100
Sue, Jones, Python 100
Larry, Larson, Python 100
--------------------------------------------------



---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------


Enter your menu choice number: 3

--------------------------------------------------
Bob, Smith, Python 100
Sue, Jones, Python 100
Larry, Larson, Python 100
--------------------------------------------------



---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------


Enter your menu choice number: 4
```

Figure 4. Notepad JSON File with Four Students




```
[
  {
    "FirstName": "Bob",
    "LastName": "Smith",
    "CourseName": "Python 100"
  },
  {
    "FirstName": "Sue",
    "LastName": "Jones",
    "CourseName": "Python 100"
  },
  {
    "FirstName": "Larry",
    "LastName": "Larson",
    "CourseName": "Python 100"
  },
  {
    "FirstName": "Audrey",
    "LastName": "Clark",
    "CourseName": "Python 100"
  }
]
```


Reflections

This was my first time creating classes, functions and the separation of concerns pattern. It took me a bit more time to understand how this new approach worked. It was helpful to review the course code demos and labs to understand the logic. Once I thought about it more and looked up the PyCharm warning, I created a new test file. I commented out the “menu_choice” variable and my warning went away (see Figure 5). Even when the warning was active, my program ran as expected.

Figure 5. PyCharm Warning Corrected

```
    # Repeat the following tasks
while True:
    IO.output_menu(menu=MENU)

    menu_choice = IO.input_menu_choice()
```

Summary

This assignment helped me understand a new way to structure my code to create the same Python program as Assignment 05 using functions, classes, and the separation of concerns pattern. It was interesting to compare differences in code from Assignment 05 to the restructuring that happened for Assignment 06.

Appendix: Assignment 06 Code

```
# ------------------------------------------------------------------------------------------ #
# Title: Assignment 06
# Desc: This assignment demonstrates using functions
# with structured error handling
# Change Log: (Who, When, What)
#   RRoot,1/1/2030,Created Script
#   Elise C., 3/3/2026, Created Script
# ------------------------------------------------------------------------------------------ #
import json

# Define the Data Constants
MENU: str = '''
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------
'''
# Define the Data Constants
# FILE_NAME: str = "Enrollments.csv"
FILE_NAME: str = "Enrollments.json"

# Define the Data Variables
students: list = []  # a table of student data
menu_choice: str = ''  # Hold the choice made by the user.

# Removed all of these and used them locally instead
#student_first_name: str = ''  # Holds the first name of a student entered by the user.
#student_last_name: str = ''  # Holds the last name of a student entered by the user.
#course_name: str = ''  # Holds the name of a course entered by the user.
#student_data: dict = {}  # one row of student data
#file = None  # Holds a reference to an opened file.

# Processing ------------------------------------- #
class FileProcessor:
    """
    A collection of processing layer functions that work with JSON files
```

```
    ChangeLog: (Who, When, What)
    Elise C., 3/3/2026, Created Class
    """

# When the program starts, read the file data into table
    # Extract the data from the file
    # Read from the JSON file
    @staticmethod
    def read_data_from_file(file_name: str, student_data: list):
        """
            This function reads the data from a file
            ChangeLog: (Who, When, What)
            Elise C.,3/3/2026,Created function

            :return: student data
        """
        file = None

        try:
            file = open(file_name, "r")
            student_data = json.load(file)
        except FileNotFoundError as e:
            IO.output_error_messages("JSON file must exist before running this script!", e)
        except Exception as e:
            IO.output_error_messages("There was a non-specific error!", e)
        finally:
            if file is not None and file.closed == False:
                file.close()
        return student_data

    # global file
    # global students
    @staticmethod
    def write_data_to_file(file_name: str, student_data: list):
        """ This function writes the data to a file
            ChangeLog: (Who, When, What)
            Elise C.,3/3/2026,Created function

            :return: None
```

```
        """
        file = None

        try:
            file = open(file_name, "w")
            json.dump(student_data, file, indent=2)
        except TypeError as e:
            IO.output_error_messages("Please check that the data is a valid JSON format", e)
        except Exception as e:
            IO.output_error_messages("There was a non-specific error!", e)
        finally:
            if file is not None and file.closed == False:
                file.close()

# Presentation ------------------------------------- #
class IO:
    """
    A collection of presentation layer functions that manage user input and output

    ChangeLog: (Who, When, What)
    Elise C.,3/3/2026,Created Class
    Elise C.,3/3/2026,Added menu output and input functions
    Elise C.,3/3/2026,Added a function to display the data
    Elise C.,3/3/2026,Added a function to display custom error messages
    """

    @staticmethod
    def output_error_messages(message: str, error: Exception = None):
        """ This function displays custom error messages to the user

        ChangeLog: (Who, When, What)
        Elise C.,3/3/2026,Created function

        :return: None
        """
        print(message, end="\n\n")
        if error is not None:
            print("-- Technical Error Message -- ")
            print(error, error.__doc__, type(error), sep='\n')
```

```
    @staticmethod
    def output_menu(menu: str):
        """ This function displays a menu of choices to the user

        ChangeLog: (Who, When, What)
        Elise C.,3/3/2026,Created function

        :return: None
        """
        print()
        print(menu)
        print()  # Adding extra space to make it look nicer.

    @staticmethod
    def input_menu_choice():
        """ This function gets a menu choice from the user
            ChangeLog: (Who, When, What)
            Elise C.,3/3/2026,Created function
            :return: string with the users choice
        """
        choice = "0"
        try:
            choice = input("Enter your menu choice number: ")
            if choice not in ("1","2","3","4"):  # Note these are strings
                raise Exception("Please, choose only 1, 2, 3, or 4")
        except Exception as e:
            IO.output_error_messages(e.__str__())  # Not passing the exception object to avoid the
technical message

        return choice

    @staticmethod
    def output_student_courses(student_data: list):
        """ This function displays the student data to the user

        ChangeLog: (Who, When, What)
        Elise C.,3/3/2026,Created function

        :return: None
        """
```

```
    # Process the data to create and display a custom message
    print()
    print("-" * 50)
    for student in student_data:
        print(f'{student["FirstName"]}, {student["LastName"]}, {student["CourseName"]}')
    print("-" * 50)
    print()

@staticmethod
def input_student_data(student_data: list):
    """ This function gets the first name, last name, and course from the user

    ChangeLog: (Who, When, What)
    Elise C.,3/3/2026,Created function

    :return: student data
    """

    try:
        # Input the data
        student_first_name = input("What is the student's first name? ")
        if not student_first_name.isalpha():
            raise ValueError("The first name should not contain numbers.")

        student_last_name = input("What is the student's last name? ")
        if not student_last_name.isalpha():
            raise ValueError("The last name should not contain numbers.")

        course_name = input("Please enter the name of the course: ")

        student = {"FirstName": student_first_name,
                   "LastName": student_last_name,
                   "CourseName": course_name}
        student_data.append(student)

    except ValueError as e:
        IO.output_error_messages("That value is not the correct type of data!", e)
    except Exception as e:
        IO.output_error_messages("There was a non-specific error!", e)
    return student_data
```

```
    #  End of function definitions

    # Beginning of the main body of this script
students = FileProcessor.read_data_from_file(file_name=FILE_NAME, student_data=students)

    # Repeat the following tasks
while True:
    IO.output_menu(menu=MENU)

    menu_choice = IO.input_menu_choice()

    if menu_choice == "1":  # Get new data (and display the change)
        students = IO.input_student_data(student_data=students)
        continue

    elif menu_choice == "2":  # Display current data
        IO.output_student_courses(student_data=students)
        continue

    elif menu_choice == "3":  # Save data in a file and Display data
        FileProcessor.write_data_to_file(file_name=FILE_NAME, student_data=students)
        IO.output_student_courses(student_data=students)
        continue

    elif menu_choice == "4":  # End the program
        break  # out of the while loop
```